CHAPTER 05

Business Applications of Linear Programming

Five applications: LP formulation, CVXPY solutions, and interpretation

TEXTBOOK	CHAPTER	WEEK
Modern Management Science, 9th ed.	Ch. 5	Week 6

Management Science · Fall 2026 · Kitae Kim, School of Business, Korea Aerospace University

Content prepared by the instructor; slide layout and formatting assisted by Codex (OpenAI).

LP Applications by Industry and Function

Industry	Problem examples	Selected example
Finance and investment	Cash-flow matching, portfolio selection, scenario-based investment planning	Portfolio selection
Marketing and sales	Advertising allocation, sales-channel planning, survey scheduling	Advertising allocation
Manufacturing	Resource transfers, make-or-buy decisions, production and inventory planning	Make-or-buy decisions
Services and workforce	Hourly staff scheduling, temporary staffing	Hourly staff scheduling
Airlines	Ticket allocation across routes and fare classes	Ticket allocation

A Real Investment Portfolio

Vanguard LifeStrategy 60% Equity Fund · Actual holdings · 31 August 2026

Underlying funds and their proportion of the portfolio

Equity funds	
Vanguard FTSE Developed World ex-U.K. Equity Index Fund	33.1%
Vanguard FTSE U.K. All Share Index Unit Trust	12.4%
Vanguard U.S. Equity Index Fund	7.7%
Vanguard Emerging Markets Stock Index Fund	4.9%
Vanguard FTSE Developed Europe ex-U.K. Equity Index Fund	1.3%
Vanguard Japan Stock Index Fund	0.7%
Vanguard Pacific ex-Japan Stock Index Fund	0.3%

Bond funds	
Vanguard Global Bond Index Fund	32.6%
Vanguard U.K. Government Bond Index Fund	3.7%
Vanguard U.K. Investment Grade Bond Index Fund	1.5%
Vanguard U.K. Inflation-Linked Gilt Index Fund	1.4%
Vanguard Global Aggregate Bond UCITS ETF	0.4%

https://fund-docs.vanguard.com/LifeStrategy_60_Equity_Fund_9241_GBP_EN_UK.pdf

Investment weights turn a portfolio into an allocation decision.

Why does this fund spread its money across many investments?

Source: Vanguard, fund factsheet, 31 August 2026, p. 2.

Portfolio Selection: Problem and Data

Allocate one unit of investment across four assets to maximize annual return.

Asset	Variable	Annual return	Risk index
Equity fund (Stocks)	S	10%	80
Bonds	B	3%	20
Income fund	I	4%	30
Money market fund	M	1%	0

Policy limits

Weighted risk ≤ 40. Equity ≤ 75%. Income fund ≥ bonds.
Money market fund: 10% to 30% of the portfolio.

Portfolio Selection: LP Model

S, B, I, M are fractions of the total investment.

Maximize $0.1S + 0.03B + 0.04I + 0.01M$

Subject to

$$S + B + I + M = 1$$

$$80S + 20B + 30I \le 40$$

$$S \le 0.75$$

$$I - B \ge 0$$

$$M \ge 0.1$$

$$M \le 0.3$$

$$S, B, I, M \ge 0$$

Portfolio Selection: CVXPY Model

```
import cvxpy as cp

# 1. Decision variables
S = cp.Variable()
B = cp.Variable()
I = cp.Variable()
M = cp.Variable()

# 2. Objective function
objective = cp.Maximize(0.1*S + 0.03*B + 0.04*I + 0.01*M)

# 3. Constraints
constraints = [
    S + B + I + M == 1,
    80*S + 20*B + 30*I <= 40,
    S <= 0.75,
    I - B >= 0,
    M >= 0.1,
    M <= 0.3,
    S >= 0, B >= 0, I >= 0, M >= 0,
]
```

Portfolio Selection: CVXPY Solution

INPUT

```
# 4. Formulate and solve the problem
problem = cp.Problem(objective, constraints)
problem.solve()

# 5. Print the results
print("Status:", problem.status)
print("Objective:", round(100 * problem.value, 1), "% annual return")
print("S:", round(100 * float(S.value), 1), "%")
print("B:", round(100 * float(B.value), 1), "%")
print("I:", round(100 * float(I.value), 1), "%")
print("M:", round(100 * float(M.value), 1), "%")
```

OUTPUT

```
Status: optimal
```

Asset	Allocation
Equity fund	40.9%
Bonds	14.5%
Income fund	14.5%
Money market fund	30%

Annual return ≈ 5.4%

Rounded shares may not total 100%.

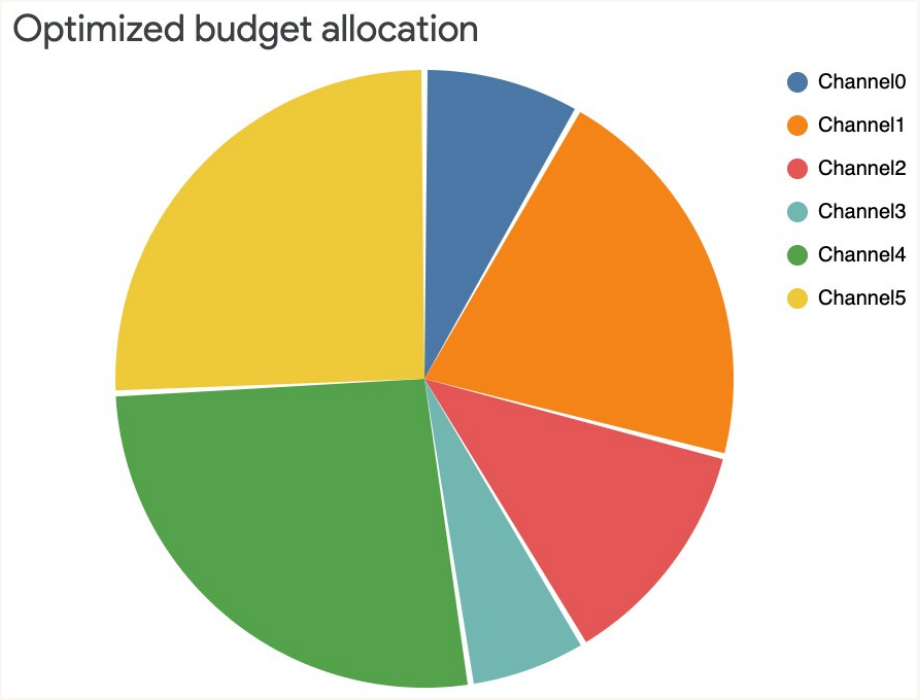
The risk limit binds at 40. The money market allocation reaches its 30% cap.

Advertising Budget Allocation in Meridian

Google Meridian · Official solution example

Optimized budget allocation

Channel	Non-optimized spend	Optimized spend
Channel_4	37%	27%
Channel_5	20%	26%
Channel_1	16%	21%
Channel_2	13%	12%
Channel_0	9%	8%
Channel_3	5%	6%



https://developers.google.com/meridian/docs/post-modeling/interpret-optimizations

Budget shares before and after optimization

Which channels should receive more of the same total budget?

Source: Google Meridian documentation. CC BY 4.0, unless otherwise noted.

Advertising Allocation: Problem and Data

Allocate advertising across TV, radio, and newspapers to maximize exposure.
Later placements have lower exposure and customer acquisition effects.

Placement block	Exposure / ad	New customers / ad	Cost / ad (KRW 10,000)
TV: first 10	90	4,000	1,000
TV: additional	55	1,500	1,000
Radio: first 15	25	2,000	300
Radio: additional	20	1,200	300
Newspaper: first 20	10	1,000	100
Newspaper: additional	5	800	100

Advertising Allocation: Variables and Policies

Placement block	Exposure / ad	New customers / ad	Cost / ad (KRW 10,000)
TV: first 10	90	4,000	1,000
TV: additional	55	1,500	1,000
Radio: first 15	25	2,000	300
Radio: additional	20	1,200	300
Newspaper: first 20	10	1,000	100
Newspaper: additional	5	800	100

Campaign requirements

Budget ≤ KRW 300 million. New customers ≥ 100,000.
TV: 14–20 placements. Radio: at most 33 and at least twice TV.
Newspaper: at least 30 placements.

Advertising Allocation: LP Model

Maximize $90T_1 + 55T_2 + 25R_1 + 20R_2 + 10N_1 + 5N_2$

Subject to

$$1{,}000T_1 + 1{,}000T_2 + 300R_1 + 300R_2 + 100N_1 + 100N_2 \le 30{,}000$$

$$4{,}000T_1 + 1{,}500T_2 + 2{,}000R_1 + 1{,}200R_2 + 1{,}000N_1 + 800N_2 \ge 100{,}000$$

$$R_1 + R_2 \ge 2T_1 + 2T_2$$

$$T_1 + T_2 \ge 14$$

$$T_1 + T_2 \le 20$$

$$R_1 + R_2 \le 33$$

$$N_1 + N_2 \ge 30$$

$T_1 \le 10$ $R_1 \le 15$ $N_1 \le 20$

All variables ≥ 0

Advertising Allocation: CVXPY Setup

```
import cvxpy as cp

# 1. Decision variables
T1 = cp.Variable()
T2 = cp.Variable()
R1 = cp.Variable()
R2 = cp.Variable()
N1 = cp.Variable()
N2 = cp.Variable()

# 2. Objective function
objective = cp.Maximize(
    90*T1 + 55*T2 + 25*R1 + 20*R2 + 10*N1 + 5*N2
)
```

Each variable represents one placement block.
The upper bounds on T1, R1, and N1 preserve the first-block limits.

Advertising Allocation: CVXPY Constraints

```
# 3. Constraints
constraints = [
    1000*T1 + 1000*T2 + 300*R1 + 300*R2 + 100*N1 + 100*N2 <= 30000,
    4000*T1 + 1500*T2 + 2000*R1 + 1200*R2 + 1000*N1 + 800*N2 >= 100000,
    R1 + R2 >= 2*T1 + 2*T2,
    T1 + T2 >= 14,
    T1 + T2 <= 20,
    R1 + R2 <= 33,
    N1 + N2 >= 30,
    T1 <= 10,
    R1 <= 15,
    N1 <= 20,
    T1 >= 0, T2 >= 0, R1 >= 0, R2 >= 0,
    N1 >= 0, N2 >= 0,
]
```

Advertising Allocation: CVXPY Solution

INPUT

```
# 4. Formulate and solve the problem
problem = cp.Problem(objective, constraints)
problem.solve()

# 5. Print the results
print("Status:", problem.status)
print("Exposure:", round(problem.value))
print("T1:", round(float(T1.value)))
print("T2:", round(float(T2.value)))
print("R1:", round(float(R1.value)))
print("R2:", round(float(R2.value)))
print("N1:", round(float(N1.value)))
print("N2:", round(float(N2.value)))
```

OUTPUT

```
Status: optimal
```

Medium	First block	Additional	Total
TV	10	6	16
Radio	15	18	33
Newspaper	20	16	36

Maximum exposure = 2,272

The plan uses the full KRW 300 million budget and attracts 134,150 customers.

MANUFACTURING

Make or Buy? A Component Cost Comparison

Thermal Mugs · 120,000 lids · OpenStax teaching example

	Relevant Costs	
	Make Internally	Buy from Plato
Direct materials	$ 0.87	
Direct labor	0.45	
Variable overhead	0.36	
Total unit relevant cost	1.68	$ 1.75
Units required	120,000	120,000
Total relevant costs	$201,600	$210,000

https://openstax.org/books/principles-managerial-accounting/pages/10-3-evaluate-and-determine-whether-to-make-or-buy-a-component

Make: $201,600 Buy: $210,000 Savings from making: $8,400

What changes when production capacity is limited?

OpenStax · Franklin, Graybeal & Cooper (2019) · CC BY-NC-SA · Unchanged source image

MANUFACTURING

Make-or-Buy Decisions: Problem and Data

Meet product demand using internal production and outside purchases at minimum cost.

Product	Demand	Make cost (KRW / unit)	Buy cost (KRW / unit)
A	60,000	4,500	6,000
B	40,000	5,000	7,000
C	35,000	2,500	5,000

Process	A (min / unit)	B (min / unit)	C (min / unit)	Capacity (min)
Manufacturing	2	3	4	216,000
Assembly	1.5	2	3	150,000
Quality control	2	1.5	4	180,000

Make-or-Buy Decisions: LP Model

A, B, C: quantities made. BA, BB, BC: quantities bought.

Minimize $4{,}500A + 5{,}000B + 2{,}500C + 6{,}000BA + 7{,}000BB + 5{,}000BC$

Subject to

$$2A + 3B + 4C \le 216{,}000$$

$$1.5A + 2B + 3C \le 150{,}000$$

$$2A + 1.5B + 4C \le 180{,}000$$

$$A + BA \ge 60{,}000$$

$$B + BB \ge 40{,}000$$

$$C + BC \ge 35{,}000$$

$$A, B, C, BA, BB, BC \ge 0$$

Make-or-Buy Decisions: CVXPY Model

```
import cvxpy as cp

# 1. Decision variables
A = cp.Variable()
B = cp.Variable()
C = cp.Variable()
BA = cp.Variable()
BB = cp.Variable()
BC = cp.Variable()

# 2. Objective function
objective = cp.Minimize(
    4500*A + 5000*B + 2500*C + 6000*BA + 7000*BB + 5000*BC
)

# 3. Constraints
constraints = [
    2*A + 3*B + 4*C <= 216000,
    1.5*A + 2*B + 3*C <= 150000,
    2*A + 1.5*B + 4*C <= 180000,
    A + BA >= 60000,
    B + BB >= 40000,
    C + BC >= 35000,
    A >= 0, B >= 0, C >= 0, BA >= 0,
    BB >= 0, BC >= 0,
]
```

Make-or-Buy Decisions: CVXPY Solution

INPUT

```
# 4. Formulate and solve the problem
problem = cp.Problem(objective, constraints)
problem.solve()

# 5. Print the results
print("Status:", problem.status)
print("KRW million:", round(problem.value))
print("A:", (round(float(A.value))))
print("B:", (round(float(B.value))))
print("C:",(round(float(C.value))))
print("BA:", (round(float(BA.value))))
print("BB:", (round(float(BB.value))))
print("BC:", (round(float(BC.value))))
```

OUTPUT

```
Status: optimal
```

Product	Make	Buy
A	54,646	5,354
B	34,016	5,984
C	0	35,000

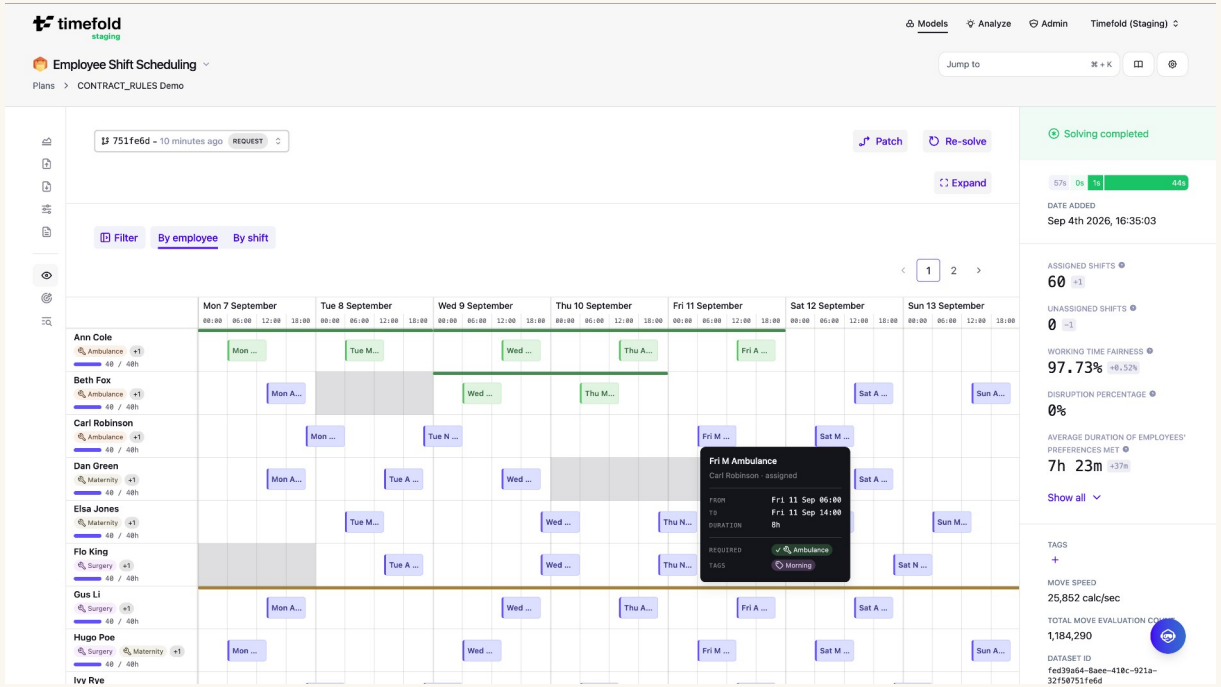
Minimum total cost = KRW 665 million

Rounded results from the notebook run.

Costs and capacity usage refer to the unrounded LP solution.

An Employee Shift Schedule

Timefold · Official employee scheduling demo



https://docs.timefold.ai/employee-shift-scheduling/latest/user-guide/visualizations

Rows identify employees. Blocks show assigned shifts across the week.

Can we cover every shift with the people available?

Source: Timefold, Employee Shift Scheduling documentation, By employee view.

Hourly Staff Scheduling: Variables and Coverage

Choose starting times and staff types to meet demand at minimum daily cost.
At least one X-type employee must cover every hour.

X_h = X-type employees starting at hour h

Y_h = Y-type employees starting at hour h

Start-time labels: h = 9, 10, 11, 12, 1, 2, 3, 4, 5, 6.
Labels 1–6 denote afternoon starting times.

Hour	Required staff	Hour	Required staff
9–10 a.m.	5	2–3 p.m.	6
10–11 a.m.	4	3–4 p.m.	4
11 a.m.–noon	9	4–5 p.m.	7
Noon–1 p.m.	10	5–6 p.m.	4
1–2 p.m.	9	6–7 p.m.	4

- X-type shift: four work hours, a one-hour break, then three work hours.
- Cost per employee: KRW 128,000.
- Y-type shift: four consecutive work hours. Cost: KRW 40,000.

- At 1–2 p.m., X9 is on break. At 2–3 p.m., X9 returns and X10 is on break.
- The coverage equations account for these breaks.

Hourly Staff Scheduling: LP Coverage Model

Minimize daily cost in KRW 10,000.

$$\text{Minimize} \quad 12.8(X_9 + X_{10} + X_{11} + X_{12} + X_1 + X_2 + X_3 + X_4 + X_5 + X_6) + 4(Y_9 + Y_{10} + Y_{11} + Y_{12} + Y_1 + Y_2 + Y_3 + Y_4 + Y_5 + Y_6)$$

Subject to

$$X_9 + Y_9 \geq 5$$
$$X_9 + X_{10} + Y_9 + Y_{10} \geq 4$$
$$X_9 + X_{10} + X_{11} + Y_9 + Y_{10} + Y_{11} \geq 9$$
$$X_9 + X_{10} + X_{11} + X_{12} + Y_9 + Y_{10} + Y_{11} + Y_{12} \geq 10$$
$$X_{10} + X_{11} + X_{12} + X_1 + Y_{10} + Y_{11} + Y_{12} + Y_1 \geq 9$$
$$X_9 + X_{11} + X_{12} + X_1 + X_2 + Y_{11} + Y_{12} + Y_1 + Y_2 \geq 6$$
$$X_9 + X_{10} + X_{12} + X_1 + X_2 + X_3 + Y_{12} + Y_1 + Y_2 + Y_3 \geq 4$$
$$X_9 + X_{10} + X_{11} + X_1 + X_2 + X_3 + X_4 + Y_1 + Y_2 + Y_3 + Y_4 \geq 7$$
$$X_{10} + X_{11} + X_{12} + X_2 + X_3 + X_4 + X_5 + Y_2 + Y_3 + Y_4 + Y_5 \geq 4$$
$$X_{11} + X_{12} + X_1 + X_3 + X_4 + X_5 + X_6 + Y_3 + Y_4 + Y_5 + Y_6 \geq 4$$

Hour	Required staff	Hour	Required staff
9–10 a.m.	5	2–3 p.m.	6
10–11 a.m.	4	3–4 p.m.	4
11 a.m.–noon	9	4–5 p.m.	7
Noon–1 p.m.	10	5–6 p.m.	4
1–2 p.m.	9	6–7 p.m.	4

Hourly Staff Scheduling: Core Staff Coverage

Subject to (continuted)

$$X_9 \geq 1$$
$$X_9 + X_{10} \geq 1$$
$$X_9 + X_{10} + X_{11} \geq 1$$
$$X_9 + X_{10} + X_{11} + X_{12} \geq 1$$
$$X_{10} + X_{11} + X_{12} + X_1 \geq 1$$
$$X_9 + X_{11} + X_{12} + X_1 + X_2 \geq 1$$
$$X_9 + X_{10} + X_{12} + X_1 + X_2 + X_3 \geq 1$$
$$X_9 + X_{10} + X_{11} + X_1 + X_2 + X_3 + X_4 \geq 1$$
$$X_{10} + X_{11} + X_{12} + X_2 + X_3 + X_4 + X_5 \geq 1$$
$$X_{11} + X_{12} + X_1 + X_3 + X_4 + X_5 + X_6 \geq 1$$
$$\text{All variables} \geq 0$$

Hour	Required staff	Hour	Required staff
9–10 a.m.	5	2–3 p.m.	6
10–11 a.m.	4	3–4 p.m.	4
11 a.m.–noon	9	4–5 p.m.	7
Noon–1 p.m.	10	5–6 p.m.	4
1–2 p.m.	9	6–7 p.m.	4

Hourly Staff Scheduling: CVXPY Setup

```
import cvxpy as cp

# 1. Decision variables
X9 = cp.Variable()
X10 = cp.Variable()
X11 = cp.Variable()
X12 = cp.Variable()
X1 = cp.Variable()
X2 = cp.Variable()
X3 = cp.Variable()
X4 = cp.Variable()
X5 = cp.Variable()
X6 = cp.Variable()
Y9 = cp.Variable()
Y10 = cp.Variable()
Y11 = cp.Variable()
Y12 = cp.Variable()
Y1 = cp.Variable()
Y2 = cp.Variable()
Y3 = cp.Variable()
Y4 = cp.Variable()
Y5 = cp.Variable()
Y6 = cp.Variable()
```

```
# 2. Objective function
objective = cp.Minimize(
    12.8*(X9 + X10 + X11 + X12 + X1 + X2 + X3 + X4 + X5 + X6)
    + 4*(Y9 + Y10 + Y11 + Y12 + Y1 + Y2 + Y3 + Y4 + Y5 + Y6)
)
```

The variable names match the LP.
No integer=True option is used.
These are continuous LP variables.

Hourly Staff Scheduling: Demand Constraints

```
# 3. Constraints
constraints = [
    X9 + Y9 >= 5,
    X9 + X10 + Y9 + Y10 >= 4,
    X9 + X10 + X11 + Y9 + Y10 + Y11 >= 9,
    X9 + X10 + X11 + X12 + Y9 + Y10 + Y11 + Y12 >= 10,
    X10 + X11 + X12 + X1 + Y10 + Y11 + Y12 + Y1 >= 9,
    X9 + X11 + X12 + X1 + X2 + Y11 + Y12 + Y1 + Y2 >= 6,
    X9 + X10 + X12 + X1 + X2 + X3 + Y12 + Y1 + Y2 + Y3 >= 4,
    X9 + X10 + X11 + X1 + X2 + X3 + X4 + Y1 + Y2 + Y3 + Y4 >= 7,
    X10 + X11 + X12 + X2 + X3 + X4 + X5 + Y2 + Y3 + Y4 + Y5 >= 4,
    X11 + X12 + X1 + X3 + X4 + X5 + X6 + Y3 + Y4 + Y5 + Y6 >= 4,
]
```

Hourly Staff Scheduling: Core Staff Constraints

```
constraints += [
    X9 >= 1,
    X9 + X10 >= 1,
    X9 + X10 + X11 >= 1,
    X9 + X10 + X11 + X12 >= 1,
    X10 + X11 + X12 + X1 >= 1,
    X9 + X11 + X12 + X1 + X2 >= 1,
    X9 + X10 + X12 + X1 + X2 + X3 >= 1,
    X9 + X10 + X11 + X1 + X2 + X3 + X4 >= 1,
    X10 + X11 + X12 + X2 + X3 + X4 + X5 >= 1,
    X11 + X12 + X1 + X3 + X4 + X5 + X6 >= 1,
    X9 >= 0, X10 >= 0, X11 >= 0, X12 >= 0,
    X1 >= 0, X2 >= 0, X3 >= 0, X4 >= 0,
    X5 >= 0, X6 >= 0, Y9 >= 0, Y10 >= 0,
    Y11 >= 0, Y12 >= 0, Y1 >= 0, Y2 >= 0,
    Y3 >= 0, Y4 >= 0, Y5 >= 0, Y6 >= 0,
]
```

constraints += [...] adds these requirements to the demand constraints.

Hourly Staff Scheduling: CVXPY Solution

INPUT

```
# 4. Formulate and solve the problem
problem = cp.Problem(objective, constraints)
problem.solve()

# 5. Print the results
print("Status:", problem.status)
print("KRW:", round(10000 * problem.value))
print("X9:", round(float(X9.value)))
print("X10:", round(float(X10.value)))
print("X11:", round(float(X11.value)))
print("X12:", round(float(X12.value)))
print("X1:", round(float(X1.value)))
print("X2:", round(float(X2.value)))
print("X3:", round(float(X3.value)))
print("X4:", round(float(X4.value)))
print("X5:", round(float(X5.value)))
print("X6:", round(float(X6.value)))
print("Y9:", round(float(Y9.value)))
print("Y10:", round(float(Y10.value)))
print("Y11:", round(float(Y11.value)))
print("Y12:", round(float(Y12.value)))
print("Y1:", round(float(Y1.value)))
print("Y2:", round(float(Y2.value)))
print("Y3:", round(float(Y3.value)))
print("Y4:", round(float(Y4.value)))
print("Y5:", round(float(Y5.value)))
print("Y6:", round(float(Y6.value)))
```

OUTPUT

```
Status: optimal
```

Start	X-type	Y-type
9 a.m.	1	4
10 a.m.	0	2
11 a.m.	1	2
Noon	0	1
1 p.m.	0	3
2 p.m.	0	0
3 p.m.	0	1
4 p.m.	0	2
5–6 p.m.	0	0

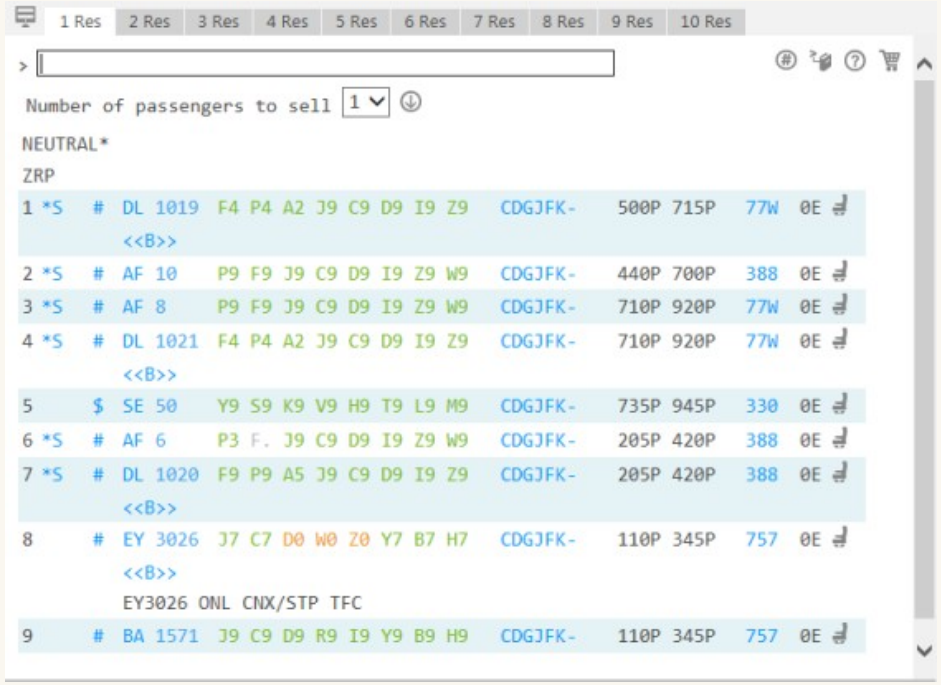
Minimum daily cost = KRW 856,000

Rounded results from the notebook run.

Rounding may violate coverage constraints.

Seats Available by Booking Class

Travelport Smartpoint · Official training example



https://support.travelport.com/webhelp/Smartpoint1P/Content/Air/AirAvailability/DisplayingAvailability.htm

Availability counts are not additive across booking classes.

Why is one booking class available while another is closed?

F4 · P4 · A2 · J9

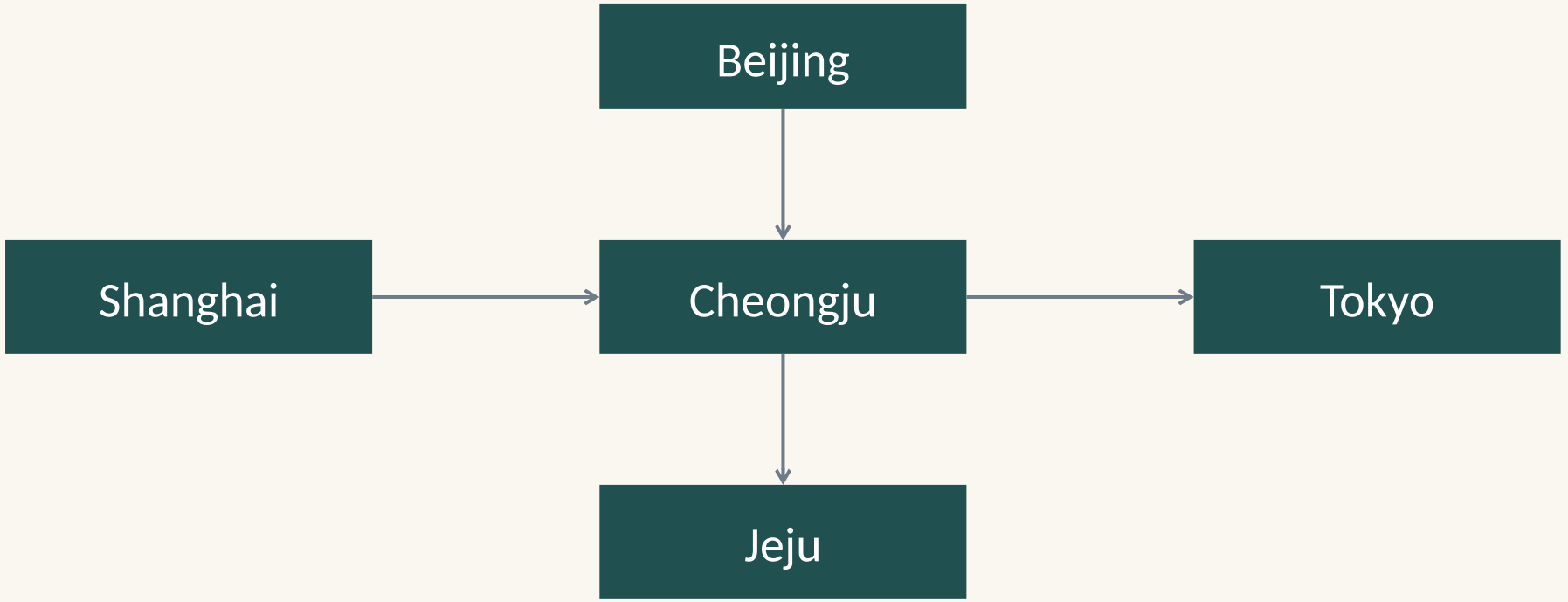
Booking codes and seats available to sell

Green: available
Grey: closed
Orange: waitlist

Source: Travelport Smartpoint, Displaying Air Availability.

Airline Ticket Allocation: Flight Network

Each flight leg has 132 seats. Connecting passengers use a seat on every leg of their route.



Airline Ticket Allocation: Fares and Demand

F = full fare. D = discount fare. Each ticket code is a decision variable.
Fares are in KRW 10,000. Demand gives an upper bound on sales.

Route	Full fare	Discount fare	Full demand	Discount demand	Variables
Beijing–Cheongju	20	13	16	33	BCF / BCD
Beijing–Jeju	30	19	6	44	BJF / BJD
Beijing–Tokyo	31	21	11	45	BTF / BTD
Shanghai–Cheongju	21	14	15	26	SCF / SCD
Shanghai–Tokyo	31	23	9	39	STF / STD
Shanghai–Jeju	29	19	7	56	SJF / SJD
Cheongju–Jeju	9	7	8	64	CJF / CJD
Cheongju–Tokyo	25	18	10	46	CTF / CTD

Airline Ticket Allocation: LP Model

Variables: the number of tickets sold in each route and fare class.

Maximize $20BCF + 13BCD + 30BJF + 19BJD + 31BTF + 21BTD$

$+ 21SCF + 14SCD + 31STF + 23STD + 29SJF + 19SJD$

$+ 9CJF + 7CJD + 25CTF + 18CTD$

Subject to

$BCF \le 16,\ BCD \le 33,\ BJF \le 6,\ BJD \le 44$

$BTF \le 11,\ BTD \le 45,\ SCF \le 15,\ SCD \le 26 \ldots$

$BCF + BCD + BJF + BJD + BTF + BTD \le 132$

$SCF + SCD + STF + STD + SJF + SJD \le 132$

$BJF + BJD + SJF + SJD + CJF + CJD \le 132$

$BTF + BTD + STF + STD + CTF + CTD \le 132$

Route	Full fare	Discount fare	Full demand	Discount demand	Variables
Beijing–Cheongju	20	13	16	33	BCF / BCD
Beijing–Jeju	30	19	6	44	BJF / BJD
Beijing–Tokyo	31	21	11	45	BTF / BTD
Shanghai–Cheongju	21	14	15	26	SCF / SCD
Shanghai–Tokyo	31	23	9	39	STF / STD
Shanghai–Jeju	29	19	7	56	SJF / SJD
Cheongju–Jeju	9	7	8	64	CJF / CJD
Cheongju–Tokyo	25	18	10	46	CTF / CTD

Airline Ticket Allocation: CVXPY Setup

```
import cvxpy as cp

# 1. Decision variables
BCF = cp.Variable()
BCD = cp.Variable()
BJF = cp.Variable()
BJD = cp.Variable()
BTF = cp.Variable()
BTD = cp.Variable()
SCF = cp.Variable()
SCD = cp.Variable()
STF = cp.Variable()
STD = cp.Variable()
SJF = cp.Variable()
SJD = cp.Variable()
CJF = cp.Variable()
CJD = cp.Variable()
CTF = cp.Variable()
CTD = cp.Variable()
```

```
# 2. Objective function
objective = cp.Maximize(
    20*BCF + 13*BCD + 30*BJF + 19*BJD + 31*BTF + 21*BTD
    + 21*SCF + 14*SCD + 31*STF + 23*STD + 29*SJF + 19*SJD
    + 9*CJF + 7*CJD + 25*CTF + 18*CTD
)
```

Each ticket has its own scalar variable. The objective adds fare × tickets sold across all 16 ticket types.

Airline Ticket Allocation: CVXPY Constraints

```
# 3. Constraints
constraints = [
    BCF <= 16,
    BCD <= 33,
    BJF <= 6,
    BJD <= 44,
    BTF <= 11,
    BTD <= 45,
    SCF <= 15,
    SCD <= 26,
    STF <= 9,
    STD <= 39,
    SJF <= 7,
    SJD <= 56,
    CJF <= 8,
    CJD <= 64,
    CTF <= 10,
    CTD <= 46,
]
```

```
constraints += [
    BCF + BCD + BJF + BJD + BTF + BTD <= 132,
    SCF + SCD + STF + STD + SJF + SJD <= 132,
    BJF + BJD + SJF + SJD + CJF + CJD <= 132,
    BTF + BTD + STF + STD + CTF + CTD <= 132,
    BCF >= 0, BCD >= 0, BJF >= 0, BJD >= 0,
    BTF >= 0, BTD >= 0, SCF >= 0, SCD >= 0,
    STF >= 0, STD >= 0, SJF >= 0, SJD >= 0,
    CJF >= 0, CJD >= 0, CTF >= 0, CTD >= 0,
]
```

Airline Ticket Allocation: CVXPY Solution

INPUT

```
# 4. Formulate and solve the problem
problem = cp.Problem(objective, constraints)
problem.solve()

# 5. Print the results
print("Status:", problem.status)
print("KRW 10000:", round(problem.value))
print("BCF:", round(float(BCF.value)))
print("BCD:", round(float(BCD.value)))
print("BJF:", round(float(BJF.value)))
print("BJD:", round(float(BJD.value)))
print("BTF:", round(float(BTF.value)))
print("BTD:", round(float(BTD.value)))
print("SCF:", round(float(SCF.value)))
print("SCD:", round(float(SCD.value)))
print("STF:", round(float(STF.value)))
print("STD:", round(float(STD.value)))
print("SJF:", round(float(SJF.value)))
print("SJD:", round(float(SJD.value)))
print("CJF:", round(float(CJF.value)))
print("CJD:", round(float(CJD.value)))
print("CTF:", round(float(CTF.value)))
print("CTD:", round(float(CTD.value)))
```

OUTPUT

```
Status: optimal
```

Route	Full	Discount
B–C	16	33
B–J	6	44
B–T	11	22
S–C	15	26
S–T	9	34
S–J	7	41
C–J	8	26
C–T	10	46

Maximum revenue ≈ KRW 66 million

All four flight legs use 132 seats.

Some discount demand remains unmet because connecting tickets share scarce seats.

SUMMARY

Five Applications of Linear Programming

Application	Decision variables	Objective	Key constraints
Portfolio selection	Investment shares	Maximize return	Budget, risk, allocation limits
Advertising allocation	Placements by medium	Maximize exposure	Budget, reach, media limits
Make-or-buy	Make and buy quantities	Minimize total cost	Capacity, product demand
Staff scheduling	Staff by starting time	Minimize staffing cost	Hourly demand, shift coverage
Airline ticket allocation	Sales by route and fare	Maximize revenue	Demand, shared flight capacity

A common workflow

Define variables, write the objective and constraints, then solve with CVXPY.

Interpret the solution in context. Rounded LP values do not enforce integer decisions.